Contrôle de TP — Programmation Python

Épreuve sur machine — Barème : 20 points

1- Déposez vos téléphones et tout appareil connecté à l'endroit indiqué par l'enseignant.

2- Toute fraude ou tentative de fraude, y compris entre étudiants, sera sanctionnée. L'étudiant qui copie et celui qui facilite la copie sont tous deux concernés.

3- Seules les constructions de base vues en cours sont autorisées. Les fonctions donnant directement le résultat (`min`, `max`, `sum`, `index`, `count`, `in` dans un test, etc.) sont interdites.

Code pour le test

Pour chaque jeu d'essai, stockez le résultat renvoyé par votre fonction dans la variable **`obtenu`** et la réponse correcte dans la variable **`attendu`**. Les valeurs à placer dans `attendu` se trouvent dans le tableau fourni pour chaque exercice. Pour chaque test, affichez les deux variables, puis le message « TEST REUSSI » ou « TEST ECHOUE » selon leur égalité. **Ce code doit être utilisé pour toutes les questions de test.**

Par exemple, si la valeur attendue de `Appel_fonction(...)` est `6` :

```
attendu = 6
obtenu = Appel_fonction(...)
print("Résultat attendu :", attendu)
print("Résultat obtenu :", obtenu)
if obtenu == attendu:
    print("TEST REUSSI")
else:
    print("TEST ECHOUE")
```

Exercice 1 — Regrouper les valeurs consécutives (5 points)

1.1 Écrire une fonction **`repetitions(L)`** qui reçoit une liste non vide et renvoie une liste de couples `[valeur, effectif]`, un pour chaque groupe de valeurs égales consécutives. Deux groupes séparés restent distincts. **(3 points)**

Exemple : `[6, 6, 1, 1, 1, 6]` donne `[[6, 2], [1, 3], [6, 1]]`.

1.2 Programmer et exécuter les quatre tests ci-dessous *(utiliser le **Code pour le test**)*. **(2 points)**

L	Résultat attendu
`[6, 6, 1, 1, 1, 6]`	`[[6, 2], [1, 3], [6, 1]]`
`[9, 9, 9]`	`[[9, 3]]`
`[4, 7, 4]`	`[[4, 1], [7, 1], [4, 1]]`
`[0]`	`[[0, 1]]`

Exercice 2 — Conserver les valeurs distinctes (5 points)

2.1 Écrire une fonction **`existe(L, x)`** qui renvoie `True` si `x` figure dans `L`, et `False` sinon, sans utiliser `in`. **(2 points)**

2.2 Écrire une fonction **`valeurs_uniques(L)`** qui construit une **nouvelle** liste sans doublons, en conservant l'ordre de première apparition. Elle doit obligatoirement appeler **`existe`**. **(2 points)**

2.3 Tester `valeurs_uniques` sur les quatre cas ci-dessous, et vérifier aussi que la liste de départ est inchangée *(utiliser le **Code pour le test**)*. **(1 point)**

L	Résultat attendu
`[8, 3, 8, 2, 3, 5]`	`[8, 3, 2, 5]`
`[6, 6, 6]`	`[6]`
`[5, 4, 3]`	`[5, 4, 3]`
`[]`	`[]`

Exercice 3 — Extrêmes et positions (5 points)

3.1 Écrire une fonction **`extremes(L)`** qui, pour une liste non vide, renvoie `[maximum, indice du premier maximum, minimum, indice du premier minimum]`. **Attention :** l'ordre demandé est le maximum puis le minimum. **(3 points)**

3.2 Écrire et exécuter les quatre tests suivants *(utiliser le **Code pour le test**)*. **(2 points)**

L	Résultat attendu
`[5, 11, 2, 11, 2]`	`[11, 1, 2, 2]`
`[-4, -2, -9]`	`[-2, 1, -9, 2]`
`[7, 7]`	`[7, 0, 7, 0]`
`[8]`	`[8, 0, 8, 0]`

Exercice 4 — Somme conditionnelle (5 points)

4.1 Écrire une fonction **`somme_intervalle(L, a, b)`** qui renvoie la somme des éléments de `L` **strictement** compris entre `a` et `b` (`a < valeur < b`). Si aucun élément ne convient, elle renvoie `0`. Ne pas utiliser `sum`. **(3 points)**

4.2 Programmer les quatre tests suivants. Attention : les bornes sont exclues *(utiliser le **Code pour le test**)*. **(2 points)**

L	a	b	Résultat attendu
`[2, 5, 8, 11, 14]`	5	14	19
`[1, 2, 3]`	4	9	0
`[-5, -2, 0, 3]`	-5	3	-2
`[6, 6, 6]`	5	7	18

Rendu

Déposer un seul fichier Python `Nom_Prenom.py` contenant les fonctions et tous les tests exécutables. Un simple commentaire avec la réponse attendue ne remplace pas un test.